

게임 서버 보안

Game Server Programming

Contents

- I. 네트워크 게임 보안 개괄
- II. 패킷 스니핑 Packet Sniffing
 - a. 중간자 공격 Man in the middle Attack
 - b. 호스트 머신상 패킷 스니핑
- III. 입력 검증
- IV. 소프트웨어 치트 감지
 - a. VAC Valve Anti-Cheat
 - b. 워든 Warden
- V. 서버 보안
 - a. 디도스 공격 DDoS Attack
 - b. 악성 데이터
 - c. 소요 시간 분석 공격
 - d. 침입
- VI. 요약



I. 네트워크 게임 보안 개괄

네트워크 게임 보안



개요:

네트워크 게임이 시작된 이래로
플레이어는 불공평한 이점을
추구(뒷장)



중요한 이유:

온라인 게임의 인기 증가
안전하고 즐거운 게임 환경을
제공하는 것의 중요성



강의 초점:

네트워크 게임의 일반적인 보안
취약성 리뷰
이러한 문제를 해결하기 위한
예방 조치에 대해 논의

대표적 불공정 행위

- **해^{Hack} 및 치트 프로그램 사용**: 자동 조준(Aimbot), 벽 뚫기(Wallhack), 무적 상태(God mode) 등을 통한 부당한 이점 획득.
- **버그 악용**: 게임 내 의도하지 않은 결함을 이용하여 점수, 아이템, 능력치 등을 비정상적으로 증가시키는 행위.
- **서버 지연^{Lag} 조작**: 고의로 지연을 발생시켜 상대방의 반응 속도를 떨어뜨리는 방식.
- **계정 공유 또는 계정 대리**: 고수 플레이어에게 계정을 맡겨 레벨, 점수, 순위를 올리는 행위.
- **매크로 및 자동화 스크립트 사용**: 반복 작업을 자동으로 수행하게 만들어 시간을 절약하고 자원을 부당하게 모으는 행위.

대표적 불공정 행위 - Cnt '

-
- **불법 거래**: 게임 아이템이나 캐릭터를 현실 화폐로 거래하여 경제적 우위를 점하는 행위.
 - **계정 해킹**: 피싱 또는 무차별 대입 공격을 통한 자격 증명 도용. 플레이어 계정에 대한 무단 액세스.
 - **데이터 가로채기**: 패킷 스니핑(Packet Sniffing)을 통해 민감한 정보를 얻음. 암호화되지 않은 통신 악용.
 - **DDoS** Distributed Denial of Service **공격**: 전술적 또는 경쟁적 이점을 얻기 위해 게임 서버를 방해.
 - **사회 공학** Social Engineering: 플레이어를 심리적으로 조종하여 민감한 정보를 공유함.

예방 조치

1. 안티 치트 메커니즘 Anti-Cheat Mechanisms :

- 비정상적인 게임 활동에 대한 실시간 모니터링
- 단순히 "프로그램을 차단"하는 수준을 넘어서, 클라이언트, 서버, 메모리, 행위 분석을 모두 활용하는 복합적인 보안 시스템
- Ex> VAC(Valve Anti-Cheat), BattlEye, Easy Anti-Cheat(EAC) 등

2. 계정 보안 프로토콜:

- 강력한 비밀번호와 다중 요소 인증을 시행
- 플레이어에게 피싱 위험에 대해 교육

3. 암호화된 통신:

- SSL/TLS를 사용하여 플레이어 데이터를 보호
 - SSL Secure Sockets Layer / TLS Transport Layer Security : 전송 계층 보안 프로토콜
- 안전한 UDP/TCP 프로토콜을 사용
 - TLS와 함께 사용, DTLS Datagram Transport Layer Security

4. 서버 보호:

- DDoS 완화 전략을 구현
- 서버 소프트웨어를 정기적으로 업데이트하고 패치

5. 플레이어 보고 시스템:

- 커뮤니티의 경계를 장려
- 의심스러운 활동을 보고하기 위한 강력한 도구를 제공

Quiz Questions

① 온라인 게임에서 일발 플레이어가 치트하는 가장 일반적인 방법은?

- A) 월해 Wall Hack
- B) 소셜 엔지니어링
- C) DDoS 공격
- D) 암호화된 통신

② 온라인 게임에서 다중 요소 인증의 목적은?

- A) 치트 소프트웨어 감지
- B) 계정 해킹 방지
- C) 서버 데이터 보안
- D) 통신 암호화

③ DDoS 공격이 온라인 게임에 문제가 되는 이유는?

- A) 플레이어 자격 증명 탈취
- B) 서버의 정상적 서비스 제공 기능을 방해 (Availability 감소)
- C) 속도 해킹을 가능하게 함
- D) 암호화 프로토콜을 손상

II. 패킷 스니핑 Packet Sniffing

1. 중간자 공격 Man-in-the-Middle Attack

2. 호스트 머신상 패킷 스니핑 Packet Sniffing on a Host Machine

패킷 스니핑

- 정의:
 - 정상적인 네트워크 작동을 넘어서 네트워크 패킷 내의 데이터를 검사하는 것을 의미
- 패킷이 이동하는 방식:
 - 패킷은 출발지에서 목적지까지 가는 길에 여러 컴퓨터(라우터 및 중개 장치)를 통과
 - 헤더 정보는 라우팅을 위해 분석되고, 헤더는 NAT를 위해 다시 작성 → 3자가 패킷을 들여다 보는 것을 막을 방법은 없음.
 - NAT (Network Address Translation): 내부 네트워크의 사설 IP 주소를 공인 IP 주소로 변환하여 인터넷과 통신할 수 있도록 하는 기능

패킷 스니핑 기술 사용 사례

- 비악용 사용 사례:
 - 심층 패킷 검사(DPI: Deep Packet Inspection):
 - 서비스 품질(QoS)에서 패킷의 내용을 기준으로 패킷의 우선순위를 지정하는 데 사용 (예: 파일 공유 데이터보다 VoIP).
- 악성 사용 사례:
 - 다음을 위해 패킷 데이터 읽기:
 - 로그인 자격 증명 훔치기
 - 온라인 게임에서 부정 행위

패킷 스니핑이 위협인 이유

1. 데이터 가로채기:

- 민감한 정보(사용자 이름, 비밀번호, 게임 데이터) 노출 가능성

2. 게임에서의 부정행위:

- 패킷 조작이나 가로채기는 불공정한 이점 제공 (불공정한 경쟁)

암호화 부족:

- 공개 데이터 전송은 공격자가 쉽게 접근할 수 있는 비밀 제공

네트워크 게임 에서 패킷 스니핑 대응

암호화:

- SSL/TLS와 같은 프로토콜을 사용하여 패킷 페이로드를 보호
- SSL (Secure Sockets Layer), TLS (Transport Layer Security): 인터넷 통신에서 데이터 보안을 위한 프로토콜. 현재 SSL은 TLS로 대체
- 민감한 데이터를 종단 간 암호화

인증 및 검증:

- 디지털 서명을 사용하여 데이터 무결성을 보장
- 클라이언트와 서버 양쪽에서 데이터 진위성을 확인

트래픽 난독화:

- 역엔지니어링을 방지하기 위해 민감한 게임 관련 데이터를 마스크

패킷 전송량/속도 제한:

- 과도한 데이터 분석을 완화하기 위해 패킷 빈도를 제한

Quiz Questions

① 악의적인 상황에서 패킷 스니핑의 주요 목적은?

- A) QoS 향상
- B) 민감한 정보 훔치기
- C) 패킷 데이터 암호화
- D) 라우팅 속도 개선

② 심층 패킷 검사는 일반적인 네트워크 작업에서 무엇을 달성하는 데 도움이 되는가?

- A) 데이터 암호화
- B) 패킷 재작성
- C) 패킷 스니핑
- D) QoS 우선순위 지정

③ 다음 중 패킷 스니핑을 방지할 수 있는 것은 무엇인가?

- A) 일반 텍스트 데이터 전송 사용
- B) 패킷 전송 속도 증가
- C) SSL/TLS 암호화 사용
- D) NAT 요구 사항 무시

II.1 중간자 공격

- 정의:
 - MITM^{Man In The Middle} 은 제3자가 두 당사자의 인지 없이 두 당사자 간의 통신을 가로채고 잠재적으로 변경할 때 발생
- 발생 방식:
 - 공공 Wi-Fi 네트워크: 패킷 가로채기에 취약
 - 공공 네트워크에서는 데이터가 암호화되지 않은 상태로 전송될 가능성이 높음.
 - 네트워크 트래픽을 가로채기 위한 도구(예: Wireshark, ARP 스푸핑)가 쉽게 사용 가능.
- 유선 네트워크:
 - 게이트웨이 머신은 맬웨어 또는 관리 감독으로 인해 패킷을 스니핑할 가능성이 있음
- ISP 수준 공격:
 - ISP 수준에 몰래 설치된 소프트웨어에 의한 가로채기
- 예:
 - 악성 Clive가 Alice와 Bob 간의 통신을 가로채서 민감한 데이터를 읽음



네트워크 게임의 MITM

1. 일반적인 대상:

- 로그인 자격 증명
- 게임 내 구매를 위한 결제 데이터

2. 잠재적 위험:

- 계정 보안 손상
- 신용카드 정보의 무단 사용
- 게임 데이터 조작

3. 주요 고려 사항:

- 로그인 자격 증명이나 결제 세부 정보와 같은 민감한 데이터는 항상 보호할 필요가 있음

MITM 공격에 맞서기

- 암호화:
 - 입증된 방법을 사용하여 민감한 데이터를 암호화
 - 로그인 자격 증명과 결제 데이터는 안전하게 전송
- 공개 키 암호화 사용:
 - 작동 방식:
 - 각 당사자는 개인 키와 공개 키를 생성
 - 공개 키는 핸드셰이크 중에 교환
 - 메시지는 수신자의 공개 키로 암호화되고 개인 키로 복호화
 - 게임의 예:
 - 클라이언트는 서버의 공개 키로 로그인 데이터를 암호화
 - 서버는 개인 키를 사용하여 복호화
- 모범 사례:
 - RSA 암호화를 위해 OpenSSL과 같은 신뢰할 수 있는 구현을 사용
 - 항상 최신의 검증된 암호화 프로토콜을 사용

RSA 암호화

1. RSA란?

1. Rivest, Shamir, Adelman(1977)이 개발
2. 큰 반소수^{semiprime} 인수분해의 어려움에 의존
 1. 반소수: 정확히 두 개의 소수의 곱으로 표현되는 자'

2. 작동 원리:

1. 공개 키: 큰 반소수
2. 개인 키: 반소수의 소인수에서 파생

3. RSA의 장점:

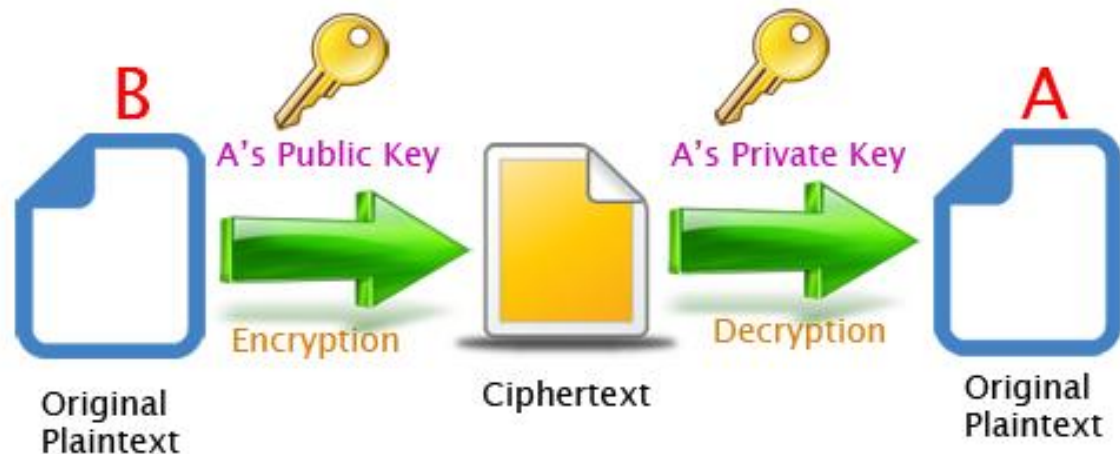
1. 잘 확립되고 안전하다고 증명
2. 계산적 불가능성으로 인해 무차별 대입 공격에 강함

4. 구현:

1. RSA 암호화를 위해 OpenSSL과 같은 오픈 소스 라이브러리를
2. 사용자가 직접 암호화 시스템을 만들려고 하지 말것

5. RSA 암호가 깨진다면...

1. RSA는 많은 보안 통신 프로토콜(예: HTTPS, SSH)의 기반
2. RSA 암호화 침해가 발생하면 전 세계적으로 거의 모든 민감한 데이터 공개



Quiz Questions

① MITM 공격자는 무엇을 하는가?

- A) 통신을 암호화
- B) 공개 키와 개인 키를 생성
- C) 네트워크 보안을 강화
- D) 동의 없이 데이터를 가로채서 읽음

② 공개 키 암호화가 MITM 공격에 효과적인 이유는 무엇인가?

- A) 핸드셰이크 프로세스를 간소화
- B) 암호화에 개인 키를 사용
- C) 의도한 수신자만 개인 키로 메시지를 해독할 수 있게 함
- D) 네트워크를 통한 데이터 전송을 방지

③ RSA 암호화의 주요 특징은 무엇인가?

- A) 대칭 키를 사용
- B) 반소수 인수분해의 어려움에 의존
- C) 공개 키 사용을 회피
- D) 외부 라이브러리 없이 구현

II.2 호스트 머신에서 패킷 스니핑

1. 정의:

- 1.호스트 머신에서 직접 패킷을 스니핑하여 허가받지 않은 정보를 얻는 행위

2.암호화가 제한되는 이유:

- 1.실행 파일에는 어떤 암호 해독 방법이 활용 되었는지의 정보 포함
- 2.결의에 가득찬 열정적인 공격자는 암호 해독 프로세스에 많은 시간과 노력을 들여 역엔지니어링이 가능할 수도 있음

• 주요 과제:

- 호스트 머신에서 패킷 스니핑에 대하여, 결의에 가득찬 열정적 공격자의 역엔지니어링에 대해 이론적으로 완벽하게 방어를 하는 궁극적 방법은 없음

호스트 기반 패킷 스니핑 완화

1. 정기적 업데이트로 암호화:

1. 암호화 키와 메모리 오프셋을 자주 변경
2. 정기적 업데이트는 해커가 성공적 치팅행위를 위해 역엔지니어링 프로세스를 계속 반복 수행하도록 강제하여 치팅 의욕을 포기하도록 유도

2. 동적 패킷 포매팅:

1. 패킷 포맷과 순서를 주기적으로 변경
2. 특정 패킷 구조에 의존하는 치팅을 쓸모없게 만듦

• 목적:

- 치트개발에 필요한 시간과 노력을 크게 늘려 해킹의욕 상실하도록 함

선택적 데이터 전송을 통한 부정행위 최소화

정보 치팅 Information Cheats:

- 공격자는 허가받지 않은 지식(예: 스텔스 플레이어의 위치)을 얻는 것을 목표

클라이언트-서버 모델 정보 치팅 방지 전략:

- 서버는 각 클라이언트에 중요하고 관련성 있는 데이터만 전송
- 예: 스텔스 모드: 스텔스 플레이어에 대한 위치 업데이트하지 않음

P2P 모델의 제한:

- 모든 관련 게임 데이터를 모든 피어에 전송해야 함
- 클라이언트-서버 모델에 비해 정보 사기를 방지하기 어려움

Quiz Questions

① 호스트 머신에서 패킷 스니핑에 대한 암호화가 완벽하게 효과적이지 않은 이유는 무엇인가?

- A) 게임에서는 암호화를 전혀 사용하지 않음
- B) 패킷 형식이 표준화되어 있음
- C) 호스트에는 패킷을 복호화할 처리 능력이 없음
- D) 이론상 역엔지니어링을 통해 복호화 프로세스를 알 수 있음

② 정보 치팅의 효과를 최소화하는 방법으로 적절한 것은?

- A) 피어 투 피어 토폴로지를 사용한다
- B) 각 클라이언트에 관련 데이터만 보낸다
- C) 패킷 형식을 정기적으로 업데이트하지 않는다
- D) 호스트 머신에서 암호화를 비활성화한다

③ 피어 투 피어 토폴로지가 치팅을 방지하는 데 있어서의 단점은 무엇인가?

- A) 모든 게임 데이터를 모든 피어와 공유한다
- B) 중요한 데이터를 단 하나의 호스트에만 보낸다
- C) 암호화 키를 자주 업데이트해야 한다
- D) 패킷 스니핑 없이는 작동할 수 없다

III. 입력검증Input Validation

입력 검증이란?

정의:

- 입력을 검증하여 네트워크 동작의 맹목적인 실행을 방지
- 입력 검증은 게임에서 플레이어가 잘못된 동작을 수행하지 않도록 보장

중요한 이유:

- 부정 행위 시도를 감지하고 방지
- 클라이언트-서버 및 피어 투 피어 게임에서 공정성을 유지

입력 검증 작동 방식

플레이어 동작 검증의 예:

- 플레이어 A가 총을 쏘기 위해 패킷 전송
- 서버측 검증 단계:
 - 플레이어 A가 무기를 가지고 있는가?
 - 무기에 총알이 있는가?
 - 무기가 쿨다운 중인가?
- 조건이 하나라도 실패하면 동작이 거부

클라이언트 소유권 검증:

- 플레이어 A를 담당하는 클라이언트가 실제로 동작을 보냈는지 확인
 - 보낸 사람이 소유하지 않은 유닛에 대한 명령을 거부

유효하지 않은 작업의 경우 처리 방법

보수적인 접근 방식:

- 플레이어를 쫓아내는 대신 잘못된 동작을 거부
- 잘못된 동작은 다음에서 발생할 수 있음을 인식
 - 지연
 - 패킷 손실

예:

- "침묵" 메커니즘이 있는 주문 시전 게임:
 - 플레이어 A가 침묵 패킷을 받기 전에 주문을 시전
 - 이 동작은 잘못되었지만 의도적인 것은 아님

플레이어를 쫓아내지 않는 이유:

- 악의적이지 않은 오류로 인해 플레이어에게 처벌을 가하는 행위 회피

Quiz Questions

- ① 게임에서 입력 검증의 목적은 무엇인가?
 - A) 그래픽 렌더링을 향상시키기 위해
 - B) 플레이어 위치를 모니터링하기 위해
 - C) 네트워크 지연 시간을 늘리기 위해
 - D) 동작이 유효하고 공정한지 확인하기 위해

- ② 잘못된 동작을 처리하는 일반적으로 적절한 방법은 무엇인가?
 - A) 문제가 있는 플레이어를 즉시 쫓아냄
 - B) 문제가 있는 플레이어의 게임을 비활성화
 - C) 잘못된 동작을 거부
 - D) 잘못된 동작을 완전히 무시

- ③ 잘못된 동작이 의도치 않게 발생할 수 있는 이유는 무엇인가?
 - A) 플레이어가 의도적으로 부정
 - B) 패킷 손실이나 지연으로 인해
 - C) 서버가 패킷을 암호화하지 못함
 - D) 잘못된 입력은 기본적으로 무시



IV. 소프트웨어 부정 탐지

Software Cheat Detection

- VAC
- Warden

부정행위에 맞서기 위한 사전 예방적 접근 방식



소프트웨어 치트 감지란?

게임 프로세스의 무결성을 적극적으로
모니터링

동일한 컴퓨터에서 실행되는 치트
소프트웨어를 감지



중요한 이유:

방어적 방법(예: 암호화, 입력 검증)은 치트
시도에만 반응

소프트웨어 치트 감지는 치트를
능동적/적극적으로 식별

치트 작동 방식1: 메모리 조작

- 게임 프로세스의 메모리를 직접 수정해 스탯 또는 상태를 변경하는 방식.
- 예시
 - FPS 게임에서 탄약 무한
 - 메모리 주소에서 Ammo = 30 값을 찾아 → 99999로 변경
 - Reload 없이 계속 발사 가능
 - RPG에서 체력(HP) 고정
 - HP 값이 줄어도 다시 원래 값으로 자동 복구되도록 메모리에 Lock
 - 레이싱 게임에서 속도 값 조작
 - Speed 변수를 강제로 500km/h로 고정
- Cheat Engine 같은 도구가 대표적

치트 작동 방식2: 외부 애플리케이션

- 게임 내부를 건드리지 않고, 밖에서 자동화하거나 입력을 조작하는 프로그램.
- 예시
 - 매크로 프로그램으로 자동 사냥
 - "10초마다 공격 버튼 → 스킬 버튼 → 이동" 반복
 - MMORPG에서 대표적인 자동 사냥 형태
 - 마우스 보정 프로그램
 - FPS에서 반동을 자동으로 잡아주는 No Recoil Tool
 - 화면 색 기반 자동 클릭기
 - 모바일 게임에서 특정 색이 나오면 자동으로 '보상 수령' 클릭
- 게임 프로세스를 건드리지 않지만 자동화로 인해 불공정.

치트 작동 방식3: 데이터 파일 수정

- 게임이 사용하는 파일을 수정하여 제한을 우회하거나 게임 규칙을 변경.
- 예시
 - 게임 설정 파일에서 시야거리/감도 제한 해제
 - config.json, .ini 파일을 직접 편집하여
 - 최대 시야거리 증가
 - 제한된 FOV(Field of View)를 150 이상으로 변경
 - 싱글 플레이 스테이터스 값 수정
 - save.dat에서 캐릭터의 돈, 자원, 레벨을 직접 변경
 - 스킨 해킹
 - 모델/스킨 파일 교체로 유료 스킨을 무료처럼 사용
- 온라인 게임은 대부분 서버 검증으로 막지만, 클라이언트 기반 게임은 여전히 취약.

치트 작동 방식4: 맵 해킹

- RTS(Real-Time Strategy) 또는 MOBA에서 전장의 안개(Fog of War) 제거.
- 예시
 - 스타크래프트 맵해킹
 - 상대 건물·유닛을 실시간으로 전부 확인
 - 드랍쉽/러쉬/확장 타이밍을 완벽히 읽음
 - 워크래프트3 맵해킹
 - 상대 영웅의 위치 추적
 - 정찰 필요 없음
 - MOBA Fog 제거 툴
 - 리그 오브 레전드·도타2에서 시야 밖의 적 유닛 위치 표시
- 실시간 전략 게임에서 가장 유명한 치트 유형.



치트 작동 방식5: 봇 (Bots)

- 봇은 사람이 하는 입력을 자동으로 수행하며 수익·아이템·정확도 등을 극대화하는 자동 플레이 도구.
- 예시
 - MMORPG 자원 채집봇
 - 광물·나무·몬스터를 자동으로 채집
 - 24시간 무한 농사 → 게임 경제 파괴
 - FPS Aim Bot(에임봇)
 - 적을 감지하면 자동으로 머리에 조준
 - 반응 시간 0ms, 헤드샷 100%
 - CClicker / 마우스 매크로 기반 자동 사냥 봇
 - 몬스터 등장 위치에 자동 스킬 반복
 - 퍼즐 게임 자동 해결 봇
 - 스크린샷 분석 → 맞는 퍼즐 선택 → 자동 클릭
- 봇은 사람보다 빠르고 정확해 게임 밸런스 붕괴의 핵심 원인.

소프트웨어 치트 감지의 장점

보이지 않는 치트를 감지

P2P 록스텝 RTS 게임의 맵 해킹
MMO 및 FPS 게임 봇

게임 무결성 강화:

공정한 플레이 보장
강력하고 신뢰할 수 있는
커뮤니티 구축

고려사항:

새로운 치트 방법에 대응하기
위해 진화할 수 있음

인기 있는 치트 탐지(Anti-Cheat) 솔루션

- 게임 파일과 프로세스의 불일치를 모니터링
- 봇과 무단 소프트웨어를 감지
- 주요 사항:
 - 보안을 유지하기 위한 제한된 공개 정보
 - 사용자 정의 구현을 위한 저수준 소프트웨어 및 리버스 엔지니어링에 대한 전문 지식이 필요



1. Valve Anti-Cheat (VAC)

- Steam 플랫폼의 대표 안티치트
- 사후 탐지(Post-detection) 방식이 강함
- 특정 시점에 치트 사용을 감지한 뒤 VAC Ban을 유저 계정에 적용
- 특징:
 - 서명 기반(sig-based) 치트 탐지
 - 메모리 스캔
 - 비인가 코드 감지
 - 커뮤니티 기반 리포트와 결합됨
 - 밴 웨이브ban wave:
 - 치팅 플레이어들을 몰아냈다가 한꺼번에 제재
 - 부정 행위자가 탐지에 빠르게 적응하는 것을 방지
- 단점: 실시간 예방력은 낮고, 탐지되면 큰 벌칙을 주는 방식

2. BattlEye (BE)

- PUBG, Escape from Tarkov, Rainbow Six Siege 등이 사용
- 매우 공격적인 실시간 탐지 시스템
- 특징:
 - 실시간 메모리 검사
 - 코드 인젝션 감지
 - 디버거/메모리 해킹 도구 차단
 - 페이로드 난독화된 치트도 비교적 잘 탐지
- 장점: 실시간 대응
- 단점: 시스템 깊게 접근 → 과거에 충돌 문제도 종종 보고됨

3. Easy Anti-Cheat (EAC)

- Fortnite, Apex Legends, Elden Ring 등 대형 타이틀에서 사용
- Epic Games에서 제공
- 특징:
 - 커널 레벨 드라이버 기반(Kernel-mode anti-cheat)
 - 메모리 변경 / 코드 인젝션 / 비정상 디버깅 탐지
 - HWID 기반 치트 사용자 추적 가능
 - 장점: 막기 어려운 커널 기반 치트에 강함
- 단점: 커널 드라이버 사용에 따른 개인정보/권한 이슈

4. GameGuard (nProtect)

- 한국 게임에서 매우 널리 사용됨 (리니지, 메이플스토리 등)
- 특징:
 - 커널 레벨 보호
 - 메모리 해킹·오토마우스 방지
 - 실행 중인 프로그램 검사
- 장점: 온라인 MMORPG에 특화
- 단점: 충돌/오탐 사례 보고가 꽤 있음

Etc.

- PunkBuster:
 - Battlefield 시리즈 초기에 유명
- XIGNCODE3:
 - 한국·아시아 지역 중심 안티치트
 - Black Desert, Sudden Attack 등에서 사용
- 게임사 자체 안티치트
 - Blizzard → Warden
 - Riot Games → Riot Vanguard
 - Activision → Ricochet
 - Ubisoft → FairFight
- 등등

소프트웨어 부정 탐지의 과제

1. 해커의 우회:

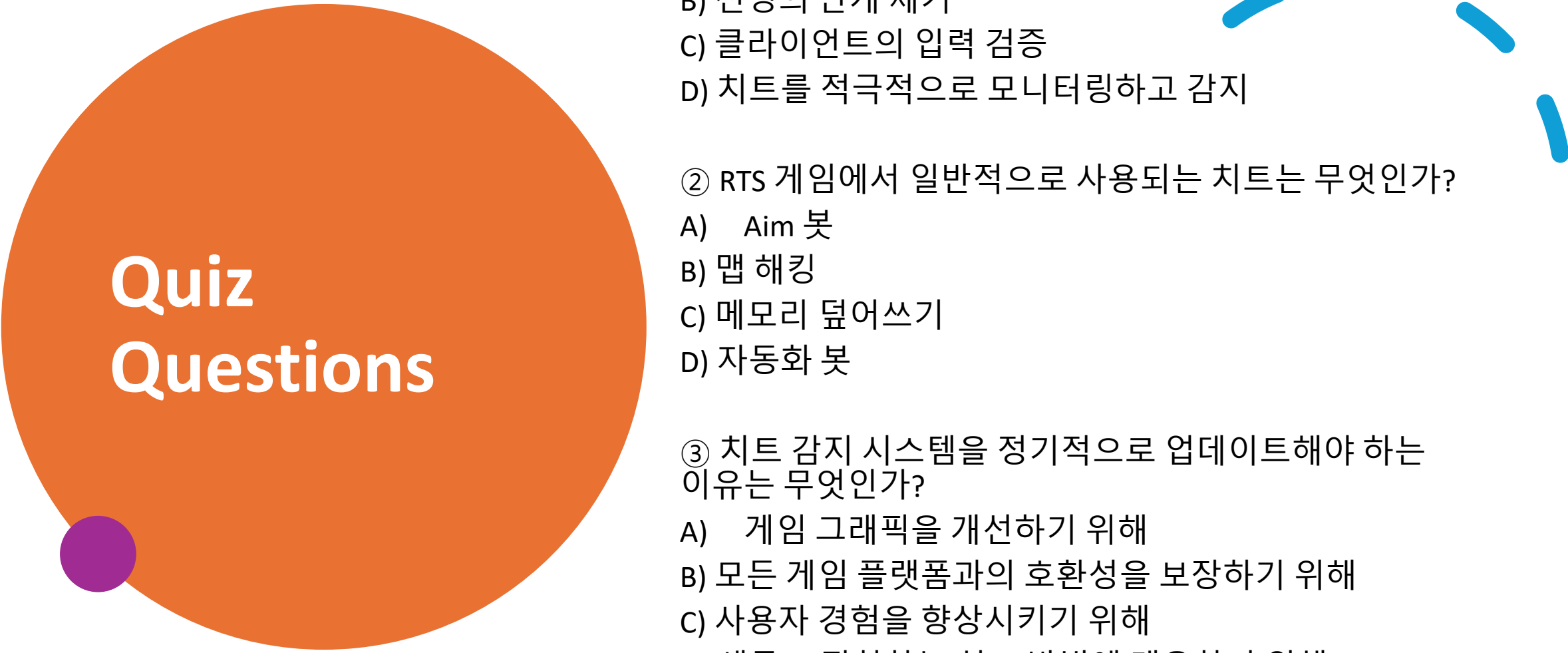
- 최고의 시스템도 충분한 노력으로 언제나 우회 가능성은 있음

2. 지속적인 업데이트 필요:

- 치트 감지는 치트 개발자보다 빠르게 발전해야 함

3. 기술적 복잡성:

- 저수준 소프트웨어 아키텍처에 대한 심층적인 지식이 필요
- 구현을 위해서는 역엔지니어링 전문 지식이 필수적



Quiz Questions

① 소프트웨어 치트 감지의 주요 목표는 무엇인가?

- A) 플레이어 데이터 암호화
- B) 전쟁의 안개 제거
- C) 클라이언트의 입력 검증
- D) 치트를 적극적으로 모니터링하고 감지

② RTS 게임에서 일반적으로 사용되는 치트는 무엇인가?

- A) Aim 봇
- B) 맵 해킹
- C) 메모리 덮어쓰기
- D) 자동화 봇

③ 치트 감지 시스템을 정기적으로 업데이트해야 하는 이유는 무엇인가?

- A) 게임 그래픽을 개선하기 위해
- B) 모든 게임 플랫폼과의 호환성을 보장하기 위해
- C) 사용자 경험을 향상시키기 위해
- D) 새롭고 진화하는 치트 방법에 대응하기 위해

IV.2 워든 Warden

1. Warden이란?

1. Blizzard Entertainment의 소프트웨어 부정 행위 탐지 프로그램
2. 모든 Blizzard 게임에서 사용

2. 핵심 기능:

1. 알려진 부정 행위에 대한 컴퓨터 메모리 및 기타 위치를 스캔
2. 탐지된 부정 행위는 Warden 서버로 보고

3. 밴 웨이브:

1. 부정 행위를 저지른 사용자는 나중에 일괄적으로 금지
2. Valve Anti-Cheat(VAC)와 유사한 접근 방식

Warden의 동적 업데이트

1. 작동 원리:

- Warden은 게임이 실행되는 동안 메모리 등 스캔
- 게임 패치 없이도 새로운 감지 기능을 자체 도입 할 수 있음

2. 치터에게 미치는 영향:

1. Warden 업데이트를 모르는 사용자를 잡음
2. 업데이트 후에도 여전히 활성화된 치트에 효과적

3. 치터들의 대응:

- 일부 치트 프로그램은 Warden 업데이트를 감지하려고 시도
- 고급 치트는 Warden이 업데이트를 완료하기 전에 스스로 언로드

워든의 장점

1. 동적이고 적응적:

- 지속적인 업데이트로 새로운 치트에 실시간으로 대응 가능
- 치트 개발자의 반응 시간을 단축

2. 통합된 접근 방식:

- 매끄러운 치트 감지를 위해 Blizzard의 생태계에 깊이 내장

3. 입증된 신뢰성:

- World of Warcraft, Overwatch, Hearthstone과 같은 Blizzard의 인기 게임에서 널리 사용

한계

1. 치터들의 대책:

- 일부 치트 프로그램은 Warden 업데이트를 감지하고 스스로 비활성화

2. 투명성:

- VAC에 비해 메커니즘에 대한 공개 정보가 적음
- 독립적인 평가가 어려움

3. 지속적인 개발 필요:

- 치터 개발자보다 앞서 나가려면 지속적인 업데이트가 필수적



V. 서버보안

Server Security

V.1 분산 서비스 거부(DDoS) 공격이란?

1.DDoS 공격의 목표:

- 과도한 요청으로 서버를 압도
- 합법적인 사용자가 서버에 접근할 수 없거나 사용할 수 없게 함

2.작동 방식:

- 네트워크 대역폭을 포화시킴
- 서버 처리 능력을 소모하여 서비스를 느리게 하거나 중단시킴

3.영향:

- 어느 시점에서는 대부분의 주요 네트워크 게임과 온라인 서비스에 영향을 미침

DDoS 공격 대비

1. 자체 호스팅 서버의 경우:

- 트래픽 관리를 위해 ISP와 긴밀히 협력
- 더 큰 부하를 처리하기 위해 서버 하드웨어를 업그레이드
- 여러 서버에 트래픽을 분산

2. 클라우드 호스팅 서버의 경우:

- 클라우드 플랫폼에서 제공하는 기본 제공 DDoS 방지 도구를 활용
- 전문 클라우드 기반 DDoS 완화 서비스를 고려

3. 주요 고려 사항:

- 클라우드 호스팅은 DDoS 위험을 줄이지만 제거하지는 않음
- 완화 전략을 정기적으로 계획하고 테스트

DDoS에 대한 사전 대책

1.트래픽 필터링:

- 방화벽과 로드 밸런서를 사용하여 악성 트래픽을 필터링

2.리소스 확장:

- 공격 중에 서버 용량을 늘리기 위해 자동 확장을 구현

3.모니터링 및 알림:

- 모니터링 시스템을 배포하여 비정상적인 트래픽 패턴을 감지하고 대응

4.협업:

- 빠른 대응을 위해 호스팅 제공업체 또는 ISP와 협력

Quiz Questions

① DDoS 공격의 주요 목적은 무엇인가?

- A) 서비스 요청으로 서버 과부하
- B) 서버 데이터 암호화
- C) 서버 성능 개선
- D) 트래픽을 다른 서버로 리디렉션

② DDoS 완화를 위한 클라우드 호스팅의 한 가지 이점은 무엇인가?

- A) 모든 공격을 완벽하게 차단
- B) 서비스 제공 업체의 내장 도구는 개발자의 부담을 줄여줌
- C) 완화 전략을 테스트할 필요가 없음
- D) 서버 대역폭 요구 사항이 줄어듦

③ 클라우드 호스팅을 사용하더라도 DDoS 공격에 대비하는 것이 중요한 이유는 무엇인가?

- A) 클라우드 공급자는 DDoS 방지를 관리하지 않음
- B) 클라우드 호스팅에서는 완화 테스트가 필요하지 않음
- C) 어떤 공급자도 완전한 보호를 보장할 수 없음
- D) 공격은 클라우드 호스팅 서버에 영향을 미치지 않음

V.2 악성데이터 Bad Data

1. Bad Data란?

- 서버로 전송된 잘못된 패킷 또는 깨졌거나 부적절한 패킷
- 서버를 크래쉬 시키거나 취약점을 악용할 수 있음

2. Bad Data 공격의 목표:

- 서버 크래시: 모든 사용자의 게임 플레이를 방해
- 악성 코드 실행: 버퍼 오버플로와 같은 취약점을 악용

악성 데이터 방어를 위한 퍼즈^{Fuzz} 테스트

1. 퍼즈 테스트란?

- 취약점을 찾기 위해 구조화되지 않은 데이터를 서버로 보내는 자동화된 테스트
 - 구조화되지 않은 데이터: 사전 정의된 데이터 모델이나 형식이 없는 데이터. 일반적으로 텍스트, 이미지, 동영상 등 다양한 형식으로 존재.

2. 목적:

- 일반적으로 표준 단위 또는 품질 보증 테스트에서 잡히지 않는 버그를 식별하고 수정
- 잠재적인 불량 데이터 공격을 시뮬레이션하여 서버 복원력을 개선
- 서버 충돌 위험 및 취약성 악용 가능성 사전 발견

Quiz Questions

① 불량 데이터 공격의 주요 목표는 무엇인가?

- A) 서버 성능을 최적화
- B) 서버 응답 시간을 개선
- C) 모든 서버 패킷을 암호화
- D) 서버를 충돌시키거나 취약점을 악용

② 퍼즈 테스트란 무엇인가?

- A) 서버 코드를 검토하는 수동 프로세스
- B) 버그를 찾기 위해 비정형 데이터를 사용하는 자동화된 테스트
- C) 네트워크 패킷을 최적화하는 방법
- D) 서버 하드웨어 업그레이드 기술

③ 퍼즈 테스트에서 구조화된 난수 데이터를 사용하는 이유는 무엇인가?

- A) 보다 현실적인 공격 시나리오를 모방
- B) 테스트 프로세스를 간소화
- C) 버그가 발견되지 않도록 보장
- D) 서버 충돌 보고서를 비활성화

V.3 타이밍 공격 (소요 시간 분석 공격)

1. 정의:

- 잘못된 데이터를 처리하고 거부하는 데 걸리는 시간의 차이를 악용하는 공격 유형

2. 악용방법:

- 공격자는 암호화 또는 해싱 알고리즘의 구현에 대해 알 수 있음
- 불일치하는 값이 거부되는 데 걸리는 시간을 관찰하여 올바른 데이터를 식별

타이밍 공격이 코드를 악용하는 방법 (취약한 함수의 예)

- 취약한 함수 샘플:
 - 8개 원소의 32비트 정수 배열로 된 인증서 두 개가 같은지 아닌지 비교하는 시나리오 →
 - a: 정답, b: 유저가 보낸 인증서
- 문제:
 - 조기 반환으로 인해 실행 시간이 달라짐:
 - 일치하지 않는 b값의 경우 더 빨리 반환.
 - 올바른 b값의 경우 더 느리게 반환.
- 공격 프로세스:
 - b[0]에 대한 모든 가능한 값을 테스트
 - 타이밍 차이를 관찰하여 올바른 값을 추론
 - 모든 인덱스에 대해 반복하여 전체 인증서를 재구성

```
bool Compare(int a[8], int b[8])
{
    for (int i = 0; i < 8; ++i)
    {
        if (a[i] != b[i])
        {
            return false;
        }
    }
    return true;
}
```

타이밍 공격 완화(타이밍 공격 저항 코드)

1. 핵심 원칙:

- 입력에 관계없이 함수가 실행되는 데 걸리는 시간이 동일하도록 구현

2. 다시 작성된 함수:

- 작동 방식:
 - 비트 XOR(^)를 사용하여 값을 비교
 - 비트 OR(|)로 결과를 결합
 - 일정한 실행 시간을 보장

```
bool Compare(int a[8], int b[8])
{
    int retVal = 0;
    for (int i = 0; i < 8; ++i)
    {
        retVal |= a[i] ^ b[i];
    }
    return retVal == 0;
}
```

Quiz Questions

① 타이밍 공격(소요시간 분석 공격)이란?

- A) 코드 실행을 최적화하는 방법
- B) 서버 응답 지연을 디버깅하는 프로세스
- C) 암호화 향상 기술
- D) 실행 시간 차이를 악용하여 정보를 추론하는 공격

② 앞에서 본 첫번째 Compare 함수에서 타이밍 취약성을 유발하는 원인은 무엇인가?

- A) XOR 연산을 수행
- B) 불일치에 대해 조기 반환을 사용
- C) 모든 인덱스를 동일하게 확인
- D) 비트 연산이 없음

③ 다시 작성된 Compare 함수는 어떻게 타이밍 공격을 완화하는가?

- A) 조기 종료를 수행
- B) 첫 번째 인덱스만 비교
- C) 일정한 실행 시간을 보장
- D) XOR 연산을 제거

V.4 침입

1. 서버 침입이란?

- 악의적인 사용자의 서버 무단 액세스

2. 목표:

- 민감한 데이터(사용자 자격 증명, 신용카드 정보)를 탈취
- 중요한 데이터베이스를 삭제하여 게임 진행 상황이나 존재 부정

3. 심각도:

- 침입은 서버 보안에 가장 심각한 결과를 초래
- 사전 예방 및 방어적 조치가 필요

침입에 대한 예방 조치

소프트웨어 업데이트 유지:

- OS, 데이터베이스, 웹 앱 및 자동화 도구에 대한 정기 업데이트
- 오래된 소프트웨어로 인한 취약성 감소

서버의 서비스 최소화:

- 실행 중인 서비스 감소 = 잠재적 침투 지점 감소

개발자 컴퓨터 보안:

- 운영 체제 및 네트워크 액세스 소프트웨어 업데이트
- 스피어 피싱 공격(spear phishing)으로부터 보호

2단계 인증 사용:

- 비밀번호를 넘어 보안 계층 추가

스피어 피싱 공격 (spear phishing): 조직에 속한 개인(들)을 속여 해당 조직에 해를 끼치는 행동을 초래하게 하는 공격

민감한 데이터 보안

1.비밀번호 보안:

- 비밀번호를 일반 텍스트로 저장하지 않기
- Bcrypt와 같은 강력한 해싱 알고리즘을 사용하기
- SHA-256, MD5 또는 DES와 같은 약한 알고리즘은 피하기

2.청구 정보 Billing info:

- 업계 모범 사례에 따라 신용 카드 데이터를 암호화 필요

3.취약성 가정:

- 잠재적 손상을 제한하기 위해 침해 발생 가정하에 대비

Bcrypt는 비밀번호를 안전하게 저장하기 위한 **해싱 알고리즘**
단방향 암호화 방식으로, **비밀번호 해싱**에 특화되어 설계됨

내부 위협에 맞서기

1. 사기 직원의 위험:

- 불만이 있는 직원은 민감한 데이터에 대한 액세스를 오용할 수 있음

2. 해결책:

- 포괄적인 로깅 및 감사 시스템을 구현
- 로그를 억제책과 법적증거로 사용

복구를 위한 백업 전략

1. 정기 백업:

- 중요한 데이터를 오프사이트 및 물리적 위치에 저장

2. 목적:

- 데이터 삭제 또는 재해 발생 시 복구를 보장

3. 중요한 이유:

- 백업에서 복원하는 것이 영구적인 데이터 손실보다 나음

Quiz Questions

① 스피어 피싱 공격에 해당하는 것은?

- A) 서버 소프트웨어를 타겟으로 하는 공격
- B) 개인 컴퓨터를 침해하여 서버에 액세스
- C) 무차별 대입 공격을 사용하여 암호를 해독
- D) 서버에 요청을 과부하

② 암호를 해싱하는 데 권장되는 알고리즘은?

- A) MD5
- B) DES
- C) SHA-256
- D) bcrypt

③ 포괄적인 로깅이 중요한 이유는?

- A) 내부 오용을 추적하고 차단
- B) 서버 부하를 줄임
- C) 민감한 데이터를 암호화
- D) 스피어 피싱 공격을 방지

Summary

- 게임 서버 보안의 주요 요점:
 - 암호화 및 테스트를 통해 데이터 전송 및 서버 무결성을 보호
 - 모든 입력을 검증하여 잘못된 작업을 방지하고 부정 행위를 줄임
 - 공정한 게임 플레이를 유지하기 위해 강력한 부정 행위 감지 시스템을 구현
- 실행 계획:
 - 정기적인 업데이트 및 비상 계획으로 사전 예방 조치를 취함
 - 클라우드 호스팅, 퍼즈 테스트 및 감사를 활용하여 보안을 강화

Review Questions

1. 중간자 공격이 실행될 수 있는 두 가지 다른 방법을 설명하시오.
2. 공개 키 암호화란 무엇인가? 이것이 중간자 공격의 위험을 최소화하는 데 어떻게 유용할 수 있는가?
3. 입력 검증이 거짓 긍정으로 이어질 수 있는 경우의 예를 들어보시오. (즉, 입력 검증에서 사용자가 부정 행위를 시도하지 않았는데도 부정 행위를 시도했다고 생각하는 경우)
4. 플레이어가 자신의 서버를 호스팅할 수 있는 게임은 서버에서 보낸 데이터를 어떻게 검증할 수 있는가?
5. 소프트웨어 부정 행위 탐지를 사용하지 않으면 록스텝 피어투피어 게임에서 맵 해킹을 감지할 수 없는 이유는 무엇인가?
6. Valve Anti-Cheat 시스템이 부정 행위를 하는 플레이어를 퇴치하는 데 어떻게 작동하는지 간략하게 설명해보시오.
7. 잠재적 침입으로부터 서버를 보호하는 두 가지 다른 방법을 설명하시오.

[EOF]
